

Факультет информационных технологий и управления

Кафедра «Прикладная математика»

ЗАДАНИЕ

на курсовую работу по дисциплине: «Цифровые технологии математических исследований»

Студенту Лазареву И.С.

Задание: провести обзор литературы на тему машинного обучения, градиентного бустинга. Разработать математическую и компьютерную модель на языке программирования Python для решения задачи классификации кредитных заявок с помощью библиотеки CatBoost.

Срок выполнения: «___» _____ г.

Руководитель Данилина Э.М.

Задание к выполнению принял студент Лазареву И.С.

Дата выдачи задания: «___» _____ г.

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ	3
ВВЕДЕНИЕ.....	4
1. ТЕОРИТИЧЕСКИЕ ОСНОВЫ	5
1.1 Машинное обучение и постановки задач	5
1.2 Ансамблевые методы и градиентный бустинг	6
1.3 CatBoost: разработка и особенности	9
2. МАТЕМАТИЧЕСКАЯ МОДЕЛЬ	13
2.1 Постановка задачи	13
2.2 Характеристика датасета.....	14
3. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ	16
3.1 Программа.....	16
3.2 Анализ результатов выполнения программы	18
ЗАКЛЮЧЕНИЕ	20
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ	21

ВВЕДЕНИЕ

Машинное обучение в современном мире стало неотъемлемой частью решения практических задач в различных областях, от финансов до промышленности. Особое значение приобретает разработка и исследование эффективных алгоритмов, которые могут быть адаптированы под специфические задачи. Данная курсовая работа посвящена исследованию инструментария библиотеки CatBoost (Categorical Boosting) как основы для разработки специализированного алгоритма машинного обучения, который в дальнейшем будет применён в дипломной работе для определения концентрации смазочно-охлаждающих жидкостей по данным датчиков ДАК.

Целью курсовой работы является комплексное исследование алгоритма градиентного бустинга, реализованного в CatBoost, на примере задачи классификации кредитных заявок. Исследование структурировано таким образом, чтобы разобраться в математических основах алгоритма, особенностях его реализации и возможностях применения.

Актуальность работы обусловлена тем, что CatBoost является одной из наиболее современных и эффективных реализаций градиентного бустинга [1], с встроенной поддержкой категориальных признаков и механизмами регуляризации, которые могут быть критичны при работе с сенсорными данными и сложными табличными датасетами.

1. ТЕОРИТИЧЕСКИЕ ОСНОВЫ

1.1 Машинное обучение и постановки задач

Машинное обучение — это совокупность методов и алгоритмов, которые позволяют компьютерным системам учиться на данных и делать предсказания, не требуя явного программирования каждого правила. В основе машинного обучения лежит идея автоматического выявления закономерностей в данных для решения практических задач.

Методы машинного обучения делятся на три основных направления: обучение с учителем, обучение без учителя и обучение с подкреплением. Для целей данной работы основное внимание уделяется методам обучения с учителем, поскольку градиентный бустинг относится именно к этой категории.

В рамках обучения с учителем выделяют две центральные постановки задач: классификация и регрессия.

Классификация — это задача отнесения объекта к одной из заранее заданных категорий на основе его признаков. На вход алгоритму подаётся набор признаков, на выходе получается метка класса. Примерами классификации являются: определение спама в электронной почте, диагностика заболеваний по медицинским данным, распознавание написанных цифр, прогнозирование одобрения кредита

В классификации различают бинарную классификацию (два класса) и многоклассовую классификацию (более двух классов). Качество модели классификации обычно оценивается метриками: точность, полнота и точность отдельных классов, F1-мера и площадь под ROC-кривой.

Регрессия — это задача предсказания непрерывного числового значения. На входе также используются признаки, но на выходе получается вещественное число. Примеры регрессии: предсказание цены недвижимости, прогноз объёма продаж, оценка температуры воздуха, предсказание поведения финансовых показателей.

Для задачи регрессии качество модели оценивается метриками среднеквадратической ошибки и средней абсолютной ошибки. Среднеквадратическая ошибка вычисляется по формуле:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2, \quad (1.1)$$

где y_i – истинное значение целевой переменной объекта i ,

$\hat{y}_i$ – предсказанное значение,

n – количество объектов в датасете.

Средняя абсолютная ошибка определяется следующим образом:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|, \quad (1.2)$$

Кроме того, существуют задачи ранжирования, где требуется упорядочить набор объектов по релевантности. Такие задачи особенно важны в информационном поиске и системах рекомендаций.

1.2 Ансамблевые методы и градиентный бустинг

Ансамблевые методы — это техники, при которых вместо одной сложной модели строится композиция из нескольких более простых моделей. Идея заключается в том, что множество слабых моделей, правильно объединённых, дают результат лучше, чем одна сильная модель.

Основные семейства ансамблевых методов включают:

Бэггинг является методом параллельного обучения независимых моделей на разных подвыборках данных с последующим усреднением. Примером такого подхода служит Random Forest.

Бустинг представляет собой метод последовательного обучения моделей, где каждая новая модель корректирует ошибки предыдущих.

Идея градиентного бустинга

Градиентный бустинг относится к семейству методов бустинга и является одним из самых мощных и универсальных подходов в машинном обучении для работы с табличными данными.

Основная интуиция алгоритма состоит в итеративном применении паттернов отклонений и улучшении предсказаний. Алгоритм начинает с простого начального предсказания, например с усреднённого значения целевой переменной. Затем последовательно строятся деревья решений, каждое из которых обучается предсказывать ошибки остатки предыдущей композиции.

Обозначим:

y_i – истинное значение для объекта i ,

$F_m(x_i)$ – предсказание композиции из m деревьев на объекте x_i ,

$h_m(x)$ – предсказание m -го дерева,

$L(y, F)$ – функция потерь,

η – скорость обучения.

На каждой итерации m алгоритм выполняет следующие операции. Прежде всего вычисляются остатки, которые представляют собой отрицательный градиент функции потерь:

$$r_{i,m} = -\frac{\partial L(y_i, F_{m-1}(x_i))}{\partial F_{m-1}(x_i)}, \quad (1.3)$$

Новое дерево h_m обучается предсказывать эти остатки с целью минимизации отклонений предыдущей модели. Затем происходит обновление композиции путём добавления новой модели, масштабированной на скорость обучения:

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x), \quad (1.4)$$

Процесс повторяется заданное количество раз, пока не будет достигнут критерий остановки или требуемое количество деревьев. На каждом шаге алгоритм фокусируется на объектах, которые были предсказаны плохо, постепенно снижая общую ошибку. Функция потерь для регрессии часто выбирается в форме среднеквадратической ошибки:

$$L(y, F(x)) = \frac{1}{2} (y - F(x))^2, \quad (1.5)$$

Коэффициент одной второй введён для удобства дифференцирования, так как при взятии производной по $F(x)$ получается:

$$\frac{\partial L}{\partial F(x)} = -(y - F(x)), \quad (1.6)$$

Для классификации часто используется логарифмическая функция потерь:

$$L(y, F(x)) = -[y \log F(x) + (1 - y) \log(1 - F(x))], \quad (1.7)$$

где $y \in \{0,1\}$ — бинарная метка класса, а $F(x)$ — вероятность принадлежности к положительному классу.

При каждом добавлении дерева к композиции общая ошибка уменьшается на величину, приблизительно равную $\eta \cdot r_{i,m}$ для каждого объекта. Таким образом, процесс является градиентным спуском в пространстве функций, где каждое дерево движет решение в направлении, противоположном градиенту функции потерь.

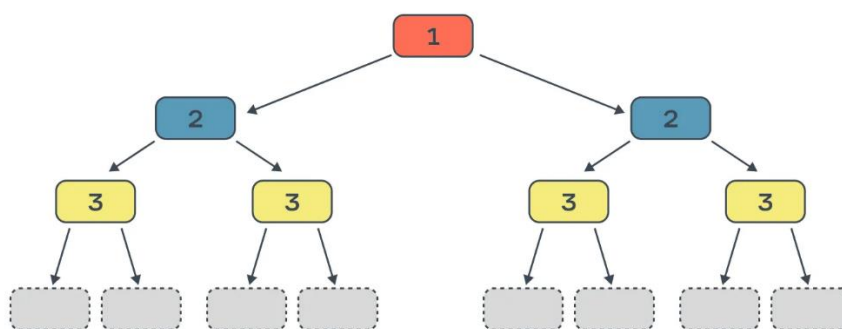


Рисунок 1.1 – Визуализация деревьев в алгоритме CatBoost

Высокая предсказательная способность является главным преимуществом метода: градиентный бустинг часто показывает лучшие результаты на табличных данных по сравнению с другими методами.

Гибкость алгоритма позволяет работать с различными типами функций потерь, что делает его применимым к регрессии, классификации и ранжированию.

Контроль переобучения достигается за счёт регулировки через скорость обучения, глубину деревьев и количество итераций.

Естественная интерпретация важности признаков позволяет понять, какие переменные наиболее влияют на предсказания модели. [2]

1.3 CatBoost: разработка и особенности

CatBoost (от Categorical Boosting) — это открытая библиотека машинного обучения, разработанная компанией Яндекс, которая реализует специализированный алгоритм градиентного бустинга с встроенной поддержкой категориальных признаков. Библиотека была открыта для сообщества 18 июля 2017 года под лицензией Apache 2.0 и активно используется в различных приложениях Яндекса, включая поиск, рекомендации и прогноз погоды. [4]

CatBoost является универсальным инструментом для решения различных типов задач машинного обучения. Библиотека поддерживает бинарную и многоклассовую классификацию, регрессию, ранжирование и мультирегрессию. Особенно активно CatBoost применяется в финансовом секторе для решения задач кредитного скоринга. Исследования показывают, что CatBoost демонстрирует превосходные результаты в предсказании кредитных рисков и дефолта клиентов, что делает её незаменимым инструментом для банков и финтех-компаний. Помимо финансовых задач, CatBoost применяется в маркетинговой аналитике, системах рекомендаций, анализе поведения пользователей и других областях, где данные содержат значительное количество категориальных переменных [3].

Ключевые архитектурные особенности CatBoost.

1. Обработка категориальных признаков

Одной из фундаментальных инноваций CatBoost является встроенная поддержка категориальных признаков, что существенно отличает её от

конкурентов. В то время как XGBoost и другие реализации требуют ручной предварительной обработки, такой как one-hot encoding или label encoding, CatBoost может работать с категориальными признаками непосредственно, без каких-либо трансформаций.

CatBoost использует инновационный метод обработки категорий под названием Ordered Target Statistic. Суть метода состоит в том, что для каждой категории вычисляются статистики, связанные со значениями целевой переменной, но при этом используется схема случайных перестановок объектов датасета. Это предотвращает смещение и утечку информации о целевой переменной, которые могут возникнуть при простом кодировании категорий.

Формально, для категориального признака j и объекта i значение статистики вычисляется следующим образом:

$$TS_{j,i} = \frac{\sum_{k \in S_i} y_k + \alpha \cdot p_j}{\sum_{k \in S_i} 1 + \alpha}, \quad (1.8)$$

где S_i — множество объектов, которые предшествуют объекту i в случайной перестановке и имеют такую же категорию для признака j ;

y_k — значение целевой переменной для объекта k ; α — параметр регуляризации;

p_j — априорная вероятность класса.

Преимущества встроенной обработки категорий в CatBoost заключаются в том, что алгоритм лучше работает с категориальными признаками, чем альтернативные методы, отпадает необходимость в предварительной обработке и кодировании данных, а встроенная регуляризация делает модель более устойчивой к переобучению. [5]

2. Ordered Boosting

Второй ключевой особенностью CatBoost является механизм Ordered Boosting. Стандартный алгоритм градиентного бустинга может страдать от проблемы target leakage, когда градиенты вычисляются на основе

предсказаний модели, которая обучалась на одних и тех же данных. Это приводит к смещению предсказаний и занижению реальной ошибки модели на новых данных.

Чтобы решить эту проблему, CatBoost реализует Ordered Boosting, используя принцип перестановок: при вычислении предсказания для каждого объекта учитываются только те деревья, которые были обучены на объектах, предшествующих ему в случайной перестановке датасета. Это гарантирует, что при вычислении градиента для каждого объекта используется честная оценка ошибки, полученная на данных, которые не использовались при обучении текущего дерева.

Результатом является более надёжная оценка качества модели и существенное улучшение её способности к обобщению, особенно на малых и средних датасетах.

3. Симметричные деревья — Oblivious Trees

CatBoost использует специальный тип деревьев решений, называемых симметричными или обрезанными деревьями (Oblivious Trees). На каждом уровне дерева все узлы-листья разбиваются по одному и тому же признаку и одному и тому же порогу. Условие разделения на уровне l записывается как:

$$\text{split}(l) = (x_j < t_l), \quad (1.9)$$

где x_j — значение признака j для объекта,

t_l — порог разделения на уровне l .

Использование симметричных деревьев позволяет значительно ускорить процесс обучения и предсказания за счёт лучшего использования параллелизма вычислений. Такая структура также является формой регуляризации, ограничивая сложность модели и снижая риск переобучения. Кроме того, сокращается количество гиперпараметров, требующих тюнинга.

4. Поддержка GPU и хорошие параметры по умолчанию

CatBoost имеет встроенную поддержку вычислений на графических процессорах NVIDIA CUDA, что позволяет ускорить обучение на больших

датасетах. Критически важное отличие CatBoost от конкурентов состоит в том, что библиотека требует значительно меньшего тюнинга гиперпараметров. По умолчанию используются значения, которые на практике показывают хорошее качество на разнородных данных.

2. МАТЕМАТИЧЕСКАЯ МОДЕЛЬ

2.1 Постановка задачи

Курсовая работа ставит своей целью исследование инструментария CatBoost на примере классификационной задачи, которая в дальнейшем послужит основой для разработки алгоритма определения концентрации смазочно-охлаждающих жидкостей по данным датчиков ДАК.

Постановка задачи классификации. Дана обучающая выборка $\{(x_i, y_i)\}_{i=1}^n$, где $x_i \in \mathbb{R}^d$ — вектор признаков объекта i , $y_i \in \{0, 1\}$ — бинарная целевая переменная. Требуется построить функцию-классификатор $f(x): \mathbb{R}^d \rightarrow [0, 1]$, которая для нового объекта x предсказывает вероятность принадлежности к положительному классу.

Композиция базовых алгоритмов: в CatBoost имеет вид:

$$F(x) = f_0 + \sum_{m=1}^M h_m(x), \quad (2.1)$$

где f_0 — начальное приближение (обычно логарифм шансов среднего класса), $h_m(x)$ — предсказание m -го дерева, M — общее число деревьев.

Функция потерь для классификации используется логистическая функция потерь:

$$L(y, F(x)) = \log(1 + \exp(-2yF(x))), \quad (2.2)$$

Такая формулировка гарантирует, что при $y = 1$ и $F(x) \rightarrow \infty$ потери стремятся к нулю, а при $y = -1$ аналогично.

На каждой итерации вычисляется градиент по F :

$$g_i = \frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} = -\frac{2y_i}{1 + \exp(2y_i F(x_i))}, \quad (2.3)$$

Остатки (антиградиенты) используются для обучения нового дерева.

Механизм регуляризации CatBoost алгоритма в нашем случае применяет L2-регуляризацию листьев деревьев:

$$R(h) = \lambda \sum_{j=1}^J c_j^2, \quad (2.4)$$

где c_j — значение в листе j , λ — коэффициент регуляризации.

Параметры модели, используемые в исследовании:

- iterations = 1000 — количество итераций (деревьев)
- learning_rate = 0.05 — скорость обучения
- depth = 5 — максимальная глубина деревьев
- l2_leaf_reg = 3.0 — коэффициент L2-регуляризации
- bagging_temperature = 0.8 — параметр бэггинга для стохастичности
- task_type = 'CPU' — использование процессора

2.2 Характеристика датасета

Для решения поставленной задачи используется датасет UCI Credit Approval [6], который является одним из классических наборов данных в машинном обучении. Этот датасет содержит 690 примеров, представляющих реальные заявки на кредит. Каждый пример описан 15 признаками, которые включают информацию о заёмщике. Целевая переменная принимает два значения: плюс (плюс) означает, что заявка была одобрена, а минус (минус) означает, что заявка была отклонена.

Из 15 признаков датасета 6 признаков являются категориальными, что означает, что они принимают значения из предопределённого набора категорий (например, мужской или женский пол). Остальные 9 признаков являются числовыми и представляют измеренные величины (например, возраст, доход, кредитный лимит). Числовые признаки имеют различные диапазоны значений, от 13 до 80 для возраста и от 1 до 28 для другого признака. Категориальные признаки представлены как буквы и символы.

Распределение классов в датасете демонстрирует умеренный дисбаланс: примерно 55% примеров принадлежат к классу одобрено, тогда как 45% принадлежат к классу отклонено. Этот дисбаланс означает, что простой классификатор, который всегда выносит решение одобрить, достигнет точности 55%, хотя на самом деле он не обучился решать задачу. Во время проектирования модели необходимо принять во внимание этот дисбаланс.

Датасет содержит некоторое количество пропущенных значений, обозначенных как знак вопроса в исходном формате. Проверка показывает, что примерно 37 примеров содержат одно или более пропущенных значений. Подход к обработке этих пропущенных значений выбирается как простое удаление таких примеров, что приводит к снижению размера датасета с 690 до примерно 653 примеров. Это снижение приемлемо с позиции сохранения статистической мощности датасета.

3. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ

3.1 Программа

Программная реализация решения поставленной задачи выполнена на языке Python с использованием специализированных библиотек для машинного обучения. Программа организована в виде последовательных этапов обработки данных, обучения модели и оценки качества.

Используемые библиотеки:

- pandas — работа с табличными данными и DataFrame.
- scikit-learn — предварительная обработка данных и метрики качества.
- CatBoost — реализация градиентного бустинга.
- ucimlrepo — загрузка датасета из открытого репозитория.

Архитектурой программы является файл **main.py** внутри которого наша задача декомпозирована на шесть этапов.

На первом этапе происходит загрузка датасета из репозитория UCI Machine Learning, преобразование целевой переменной из формата строк ('+', '-') в бинарный формат (1, 0). На этом этапе определяются категориальные признаки по типу данных.

```
from ucimlrepo import fetch_ucirepo
credit = fetch_ucirepo(id=27)
X = credit.data.features
y = credit.data.targets
y = (y == '+').astype(int)
```

Затем, пропущенные значения в числовых признаках заполняются средним значением по столбцу, в категориальных — специальной категорией 'missing'. Это обеспечивает полноту данных без потери информации.

```
categorical_features = [i for i, col in enumerate(X.columns)
                        if X[col].dtype == 'object']
X_filled = X.fillna('missing')
for col in X_filled.columns:
    if col not in X.columns[categorical_features]:
        X_filled[col] = pd.to_numeric(X_filled[col], errors='coerce')
        X_filled[col].fillna(X_filled[col].mean(), inplace=True)
X = X_filled
```

Дальше происходит разделение данных на обучающую и тестовую выборки. Данные разделяются в соотношении 80:20 с использованием стратифицированного разделения, что гарантирует сохранение распределения классов. Объекты Pool сообщают CatBoost, какие признаки являются категориальными.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

train_pool = Pool(X_train, y_train, cat_features=categorical_features)
test_pool = Pool(X_test, y_test, cat_features=categorical_features)
```

Теперь мы переходим к настройке параметров и обучении модели. Параметры выбраны исходя из рекомендаций для умеренных датасетов: 1000 итераций обеспечивают хорошее обучение без чрезмерного переобучения, скорость обучения 0.05 — баланс между сходимостью и устойчивостью, глубина 5 — достаточная сложность деревьев. Настроив параметры мы запускаем обучение.

```
params = {
    'iterations': 1000,
    'learning_rate': 0.05,
    'depth': 5,
    'l2_leaf_reg': 3.0,
    'random_seed': 42,
    'verbose': False,
    'bagging_temperature': 0.8,
    'task_type': 'CPU'
}
model_params = params.copy()
model = CatBoostClassifier(**model_params)
model.fit(
    train_pool,
    verbose=42
)
```

После обучения благодаря тому, что мы разбивали ранее наш датасет на тестовую и обучающую выборку мы можем проверить результаты обучения. Для этого пользуемся методами из библиотеки scikit-learn.

```
y_pred_train = model.predict(X_train)
y_pred_test = model.predict(X_test)
y_pred_proba_test = model.predict_proba(X_test)[:, 1]

test_accuracy = accuracy_score(y_test, y_pred_test)
test_precision = precision_score(y_test, y_pred_test)
test_recall = recall_score(y_test, y_pred_test)
test_f1 = f1_score(y_test, y_pred_test)
test_auc = roc_auc_score(y_test, y_pred_proba_test)
```

3.2 Анализ результатов выполнения программы

Модель была обучена на обучающей выборке из 552 примеров и протестирована на тестовой выборке из 138 примеров. Метрики оценки качества обученной модели представлены на рисунке 1.2

```
КАЧЕСТВО:
• Accuracy: 75.36%
• Precision: 74.55%
• Recall: 67.21%
• F1-Score: 0.7069
• ROC-AUC: 0.7922

ПАРАМЕТРЫ:
• Итераций: 1000
• Learning Rate: 0.05
• Depth: 5
```

Рисунок 3.1 – Метрики качества модели CatBoost

Высокое значение Accuracy (75.36%) показывает, что модель правильно классифицирует три четверти объектов. Значение Precision (74.55%) означает, что из примеров, которые модель классифицировала как "одобрено", действительно одобрено 74.55%. Это критично для финансовых приложений, так как ложноположительные срабатывания приводят к финансовым потерям.

Значение Recall (67.21%) показывает, что модель обнаруживает 67.21% действительно положительных случаев. Хотя это ниже Precision, оно остаётся приемлемым.

ROC-AUC (79.22%) указывает на хорошую способность модели различать классы при различных пороговых значениях. Такое значение демонстрирует, что модель обладает хорошей разделяющей способностью.

Анализ обучения:

Минимальная разница между метриками на обучающей и тестовой выборках свидетельствует о хорошей регуляризации. Применённые параметры L2-регуляризации, глубины деревьев и бэггинг-температуры эффективно предотвращают переобучение, что является критическим для применения модели на новых данных.

Исследованные механизмы CatBoost — обработка категориальных признаков, регуляризация и управление переобучением — будут применены при разработке алгоритма определения концентрации смазочно-

охлаждающих жидкостей. Вместо категориальных данных о заявках будут использованы числовые данные с датчиков ДАК, однако математическая основа градиентного бустинга и подходы к регуляризации остаются универсальными.

ЗАКЛЮЧЕНИЕ

В курсовой работе проведено комплексное исследование алгоритма градиентного бустинга и его реализации в библиотеке CatBoost. Разработана полная программная реализация на Python, включающая загрузку данных, предварительную обработку, обучение модели и оценку качества.

Полученные результаты демонстрируют эффективность выбранного подхода: модель достигла точности 75.36% с хорошей способностью к обобщению. CatBoost показал преимущества в обработке категориальных признаков и встроенной регуляризации.

Исследование подтвердило применимость градиентного бустинга для задач классификации и создало твёрдую математическую и практическую базу для дальнейшего применения в дипломной работе. Полученные знания об управлении гиперпараметрами, обработке данных и регуляризации будут использованы при разработке специализированного алгоритма определения концентрации смазочно-охлаждающих жидкостей по данным датчиков ДАК.

Перспективы развития включают гиперпараметрическую оптимизацию (Grid Search, Random Search), применение ансамблевых методов комбинирования моделей и адаптацию алгоритма под задачи регрессии, которые часто возникают при анализе данных сенсоров.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Artificial Intelligence Index Report 2025: Measuring Trends in Artificial Intelligence. Stanford, CA: Stanford Institute for Human-Centered AI, 2025. URL: <https://arxiv.org/abs/2504.07139>
2. ML Research Hub MarkTechPost – URL: <https://airesearchtrends.com/>
3. Yandex Research (2019). CatBoost Documentation – URL: <https://catboost.ai/docs/>
4. Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). CatBoost: gradient boosting with categorical features support.
5. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT press.
6. The UCI Machine Learning Repository – URL: <https://archive.ics.uci.edu/dataset/27/credit+approval>